

Software Design Document

Software Specification Document

Table of Contents

1. [Purpose](#purpose)
2. [Sequence Diagrams](#sequence-diagrams)
 - [Use Case UC1: User Login](#use-case-uc1-user-login)
 - [Use Case UC2: View Investment Portfolio](#use-case-uc2-view-investment-portfolio)
 - [Use Case UC3: Receive Investment Notifications](#use-case-uc3-receive-investment-notifications)
3. [Major Design Decisions](#major-design-decisions)
 - [Cohesion and Coupling](#cohesion-and-coupling)
 - [Non-Functional Requirements](#non-functional-requirements)
4. [Architecture](#architecture)
 - [Architectural Patterns](#architectural-patterns)
 - [System Decomposition](#system-decomposition)
 - [Component Descriptions and Interfaces](#component-descriptions-and-interfaces)
5. [Use of Design Patterns](#use-of-design-patterns)
6. [Detailed Class Diagrams](#detailed-class-diagrams)
 - [Authentication Service Classes](#authentication-service-classes)
 - [Investment Service Classes](#investment-service-classes)
 - [Notification Service Classes](#notification-service-classes)

7. [Test Driven Development](#test-driven-development)

- [Test Case TC1](#test-case-tc1)

- [Test Case TC2](#test-case-tc2)

- [Test Case TC3](#test-case-tc3)

- --

Purpose

The purpose of this document is to provide a detailed software specification for the development of a financial system that monitors investments. The system is designed to meet the following key requirements:

- **100% Availability:** The system must be operational at all times without downtime.

- **Client Data Privacy:** Ensure the confidentiality and integrity of client data.

- **Data Integrity:** Maintain accurate and consistent data throughout the system.

This document outlines the system architecture, design decisions, component interactions, and testing strategies to fulfill these requirements.

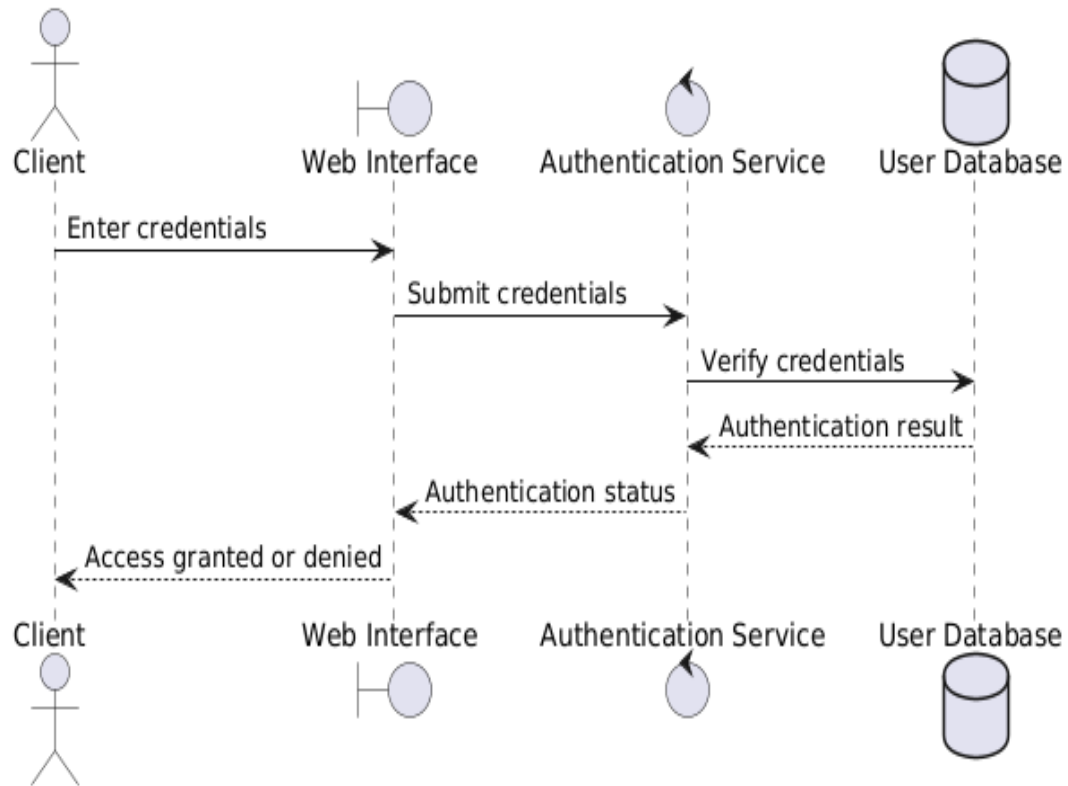
Sequence Diagrams

Use Case UC1: User Login

Description: A client logs into the system to access their investment portfolio.

Sequence Diagram UC1

``plantuml



...

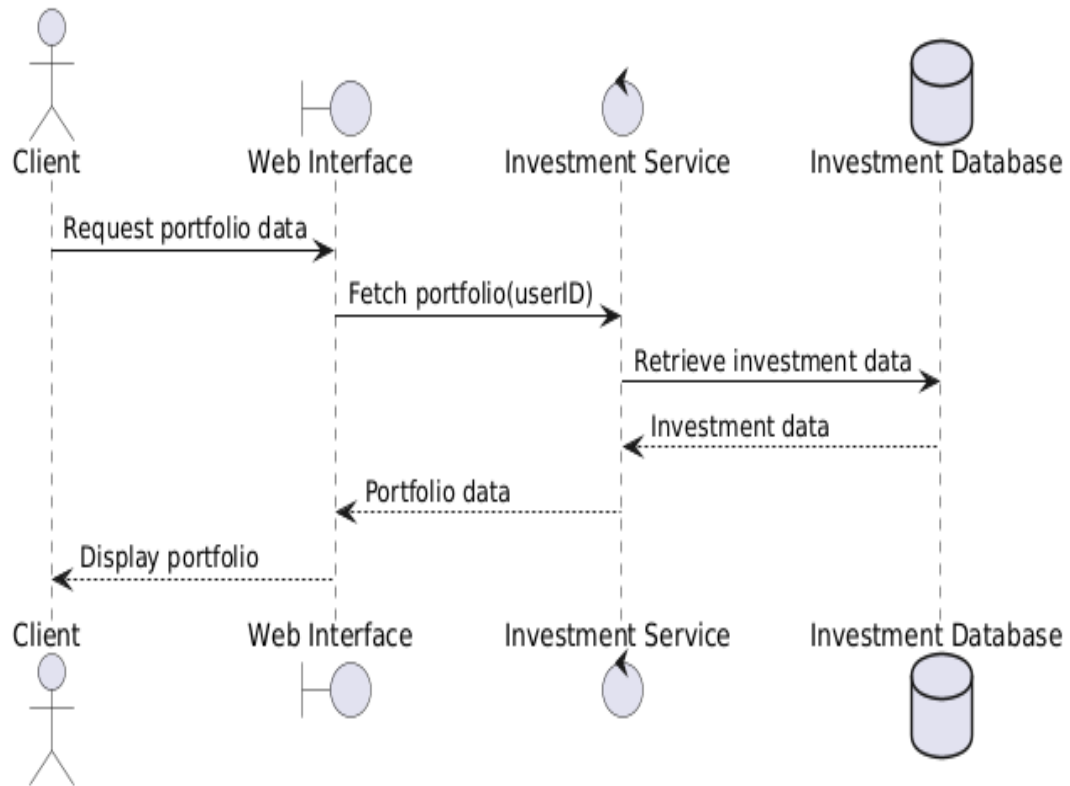
Use Case UC2: View Investment Portfolio

Description: A client views their investment portfolio in real-time.

Sequence Diagram UC2

```

sequenceDiagram
  participant Client
  participant Web Interface
  participant Authentication Service
  participant User Database
  
```



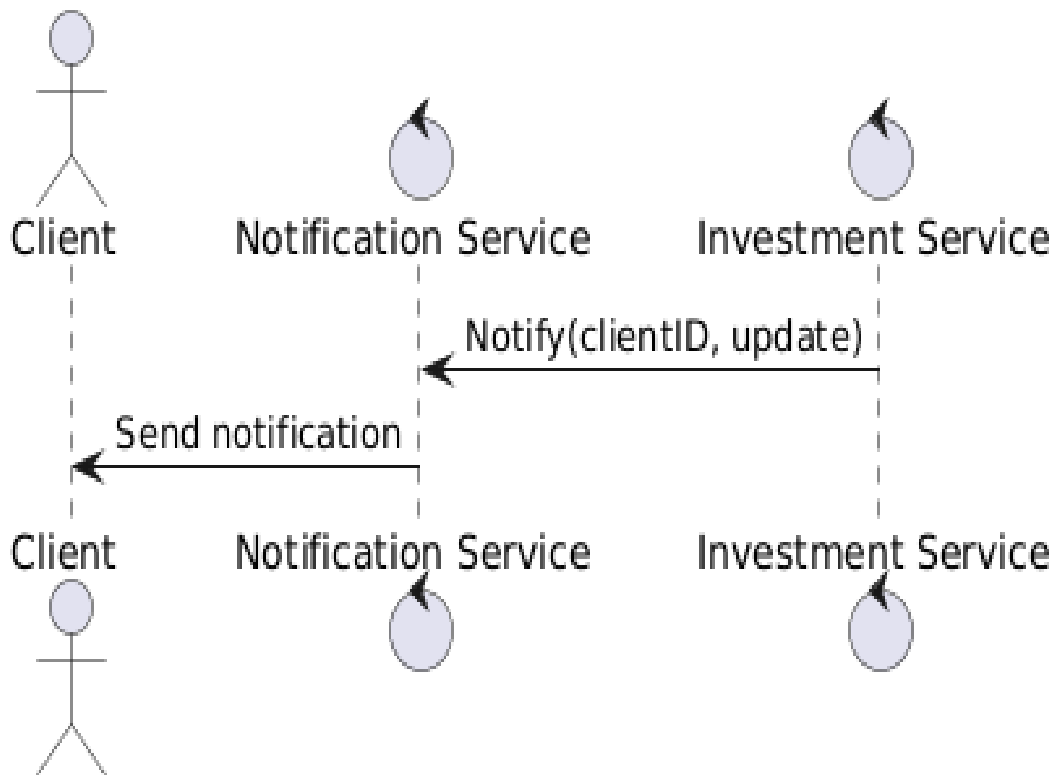
...

Use Case UC3: Receive Investment Notifications

****Description:**** A client receives notifications about their investments.

Sequence Diagram UC3

```plantuml



...

## Major Design Decisions

### Cohesion and Coupling

- **High Cohesion:** Components are organized by functionality—Authentication, Investment Management, Notification—ensuring each module has a clear responsibility.
- **Low Coupling:** Components interact through well-defined interfaces and APIs, reducing dependencies and facilitating easier maintenance and scalability.

### Non-Functional Requirements

#### 1. **Security:**

- Implement robust encryption for data at rest and in transit.
- Use multi-factor authentication for user login.
- Regular security audits and compliance checks.

## 2. **Reliability (Availability):**

- Use redundant servers and failover mechanisms.
- Implement load balancing to distribute requests evenly.
- Monitor system health continuously with automated recovery procedures.

## **Architecture**

### **Architectural Patterns**

- **Microservices Architecture:** The system is decomposed into loosely coupled services—Authentication, Investment, and Notification services—allowing independent deployment and scaling.
- **Event-Driven Architecture:** Utilized for real-time notifications, where investment updates trigger events that are consumed by the Notification Service.

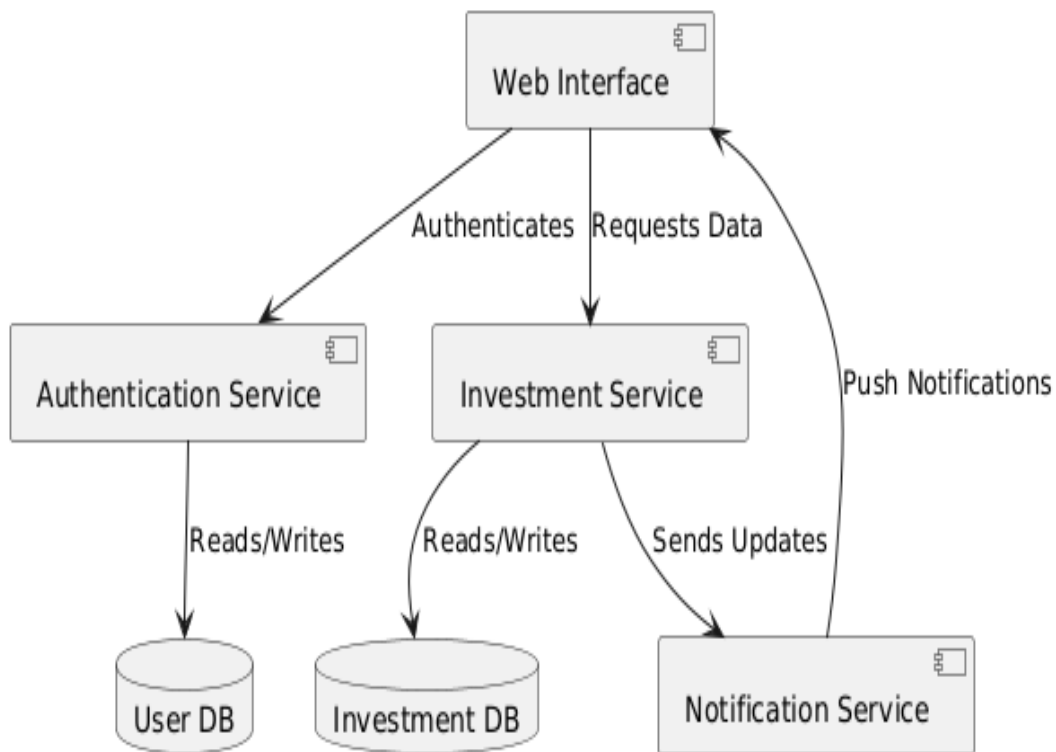
### **Rationale:**

- **Microservices:** Enhances scalability and maintainability. Services can be updated without affecting the entire system, contributing to 100% availability.
- **Event-Driven:** Supports real-time data processing and responsiveness, crucial for timely investment monitoring and notifications.

### **System Decomposition**

### **Component Diagram**

``plantuml



...

## Component Descriptions and Interfaces

### **\*\*Authentication Service (AS)\*\***

- **\*\*Functionality:\*\*** Handles user authentication and authorization.
- **\*\*Interface Operations:\*\***
  - ``login(credentials)``: Authenticates users.
  - ``logout(userID)``: Terminates user sessions.
  - ``authorize(token)``: Validates access tokens.

### **\*\*Investment Service (IS)\*\***

- **\*\*Functionality:\*\*** Manages investment data and provides portfolio information.
- **\*\*Interface Operations:\*\***

- `getPortfolio(userID)` : Returns current portfolio.
- `updateInvestments(data)` : Processes new investment data.
- `subscribeUpdates(userID)` : Registers for notifications.

### **\*\*Notification Service (NS)\*\***

- **\*\*Functionality:\*\*** Sends notifications to clients about investment updates.
- **\*\*Interface Operations:\*\***
- `sendNotification(userID, message)` : Delivers messages to clients.
- `registerDevice(userID, deviceInfo)` : Registers client devices.

### **\*\*Web Interface (WI)\*\***

- **\*\*Functionality:\*\*** Provides the user interface for clients to interact with the system.
- **\*\*Interface Operations:\*\***
- `displayPortfolio(data)` : Shows investment data to clients.
- `receiveInput(action)` : Handles user actions.

## **Use of Design Patterns**

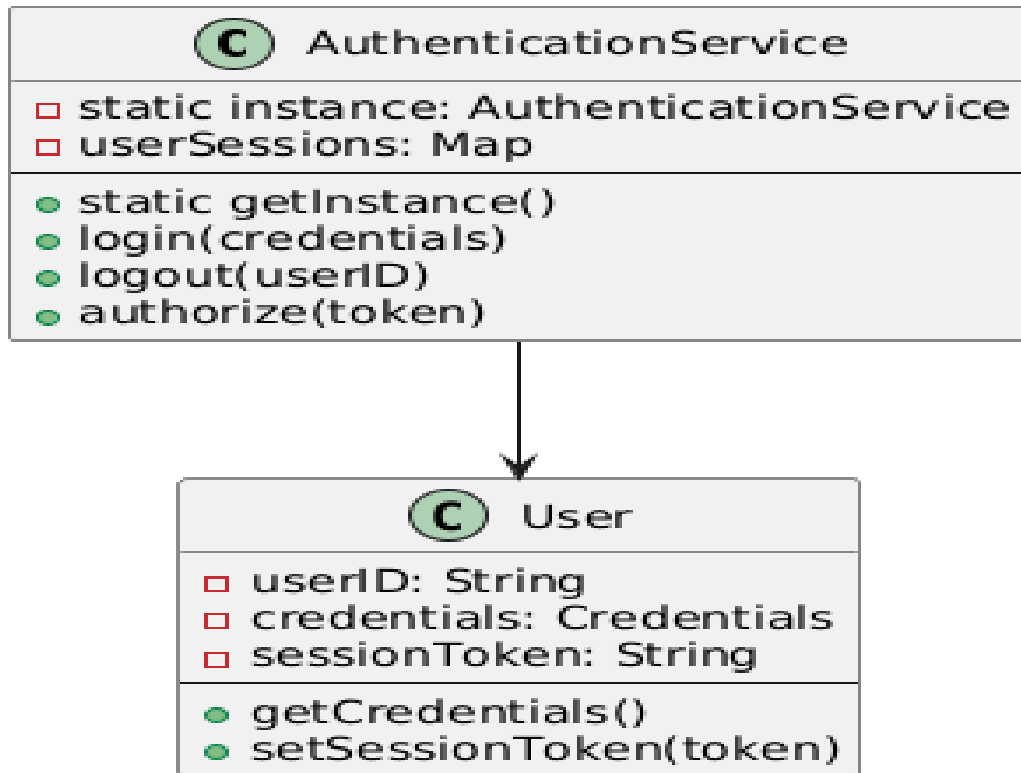
- **\*\*Singleton Pattern (Authentication Service):\*\***
- **\*\*Rationale:\*\*** Ensures a single instance of the Authentication Service manages all authentication requests, maintaining consistency in session management.
- **\*\*Observer Pattern (Notification Service):\*\***
- **\*\*Rationale:\*\*** Allows the Notification Service to subscribe to events from the Investment Service and push updates to clients in real-time.
- **\*\*Facade Pattern (Web Interface):\*\***

- **Rationale:** Provides a simplified interface to the underlying subsystem (services), making it easier for clients to interact with the system.

## Detailed Class Diagrams

### Authentication Service Classes

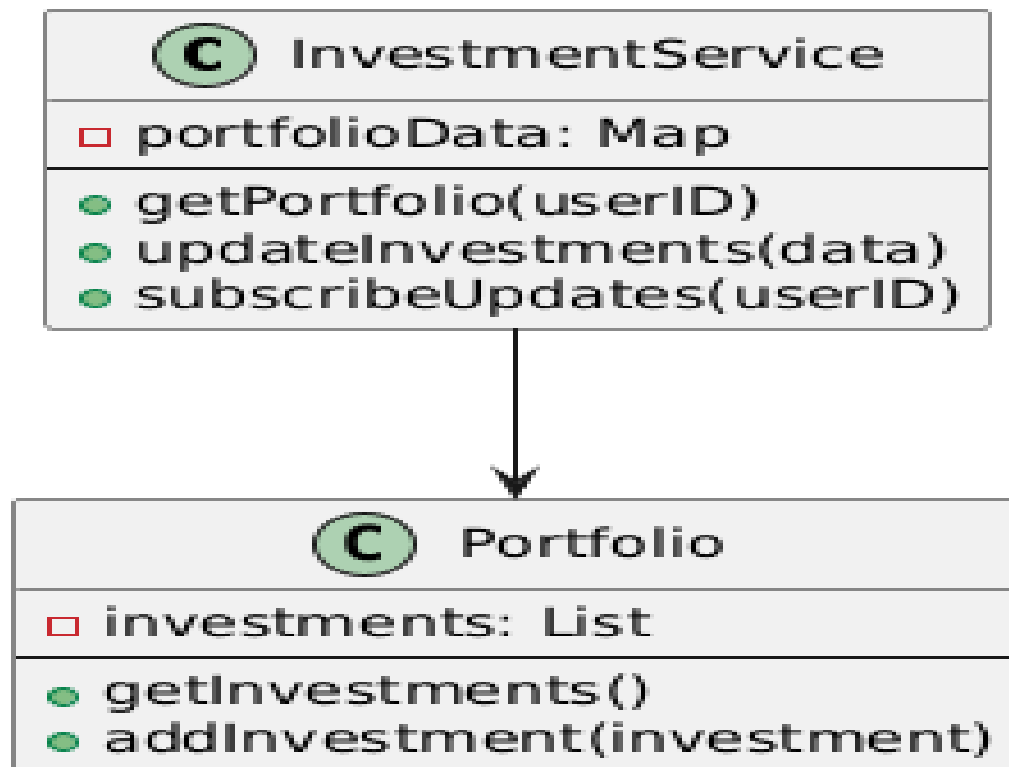
```
``plantuml
```



```
...
```

### Investment Service Classes

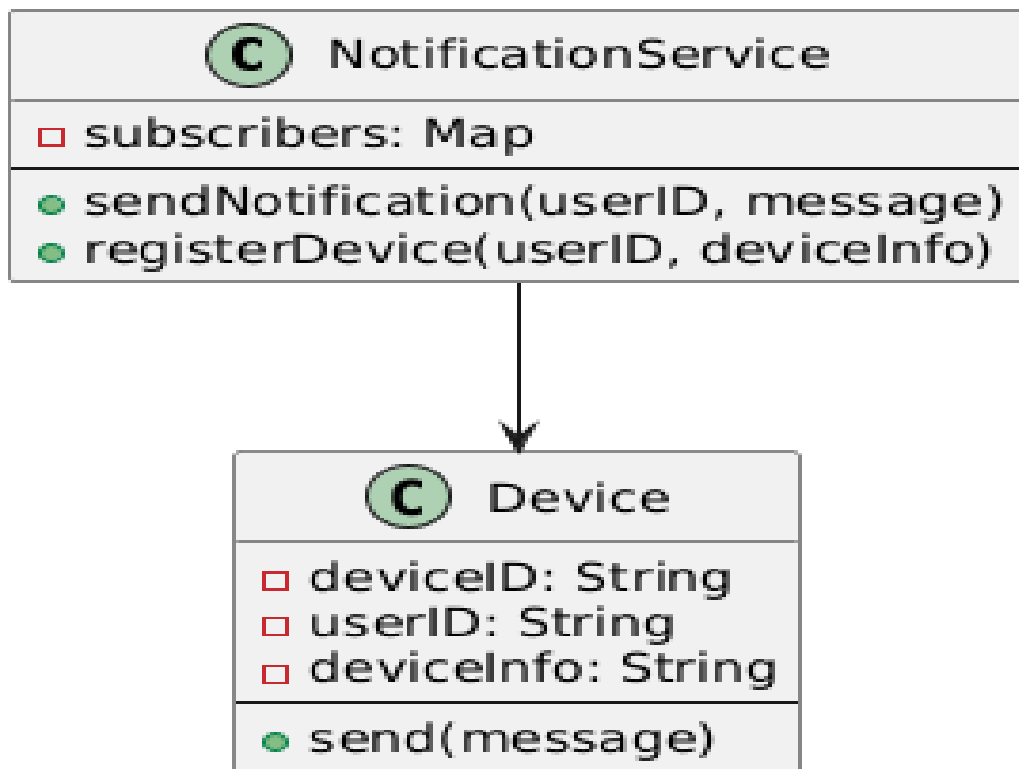
```
``plantuml
```



...

#### Notification Service Classes

```
``plantuml
```



...

## Test Driven Development

### Test Case TC1

- **Test ID:** TC1
- **Category:** User Authentication
- **Initial Condition:** User database contains valid user credentials; the Authentication Service is running.
- **Procedure:**
  1. User enters valid username and password.
  2. System submits credentials to Authentication Service.
  3. Authentication Service verifies credentials.

4. System grants access upon successful authentication.

- **Expected Outcome:** User is authenticated, and a session token is generated.

## **Test Case TC2**

- **Test ID:** TC2

- **Category:** Investment Data Retrieval

- **Initial Condition:** User is authenticated; Investment Service is up-to-date with the latest investment data.

- **Procedure:**

1. User requests their investment portfolio.
2. System calls `getPortfolio(userID)` on Investment Service.
3. Investment Service retrieves data from Investment Database.
4. System displays portfolio to the user.

- **Expected Outcome:** User receives an accurate and current view of their investment portfolio.

## **Test Case TC3**

- **Test ID:** TC3

- **Category:** Real-Time Notifications

- **Initial Condition:** User is subscribed to notifications; Notification Service is connected.

- **Procedure:**

1. An investment update occurs.
2. Investment Service triggers an event.
3. Notification Service captures the event.
4. Notification Service sends notification to the user.

- **Expected Outcome:** User receives a real-time notification about the investment update.

- --

This software specification document outlines the architecture, design decisions, and testing strategies for developing a robust, secure, and highly available financial investment monitoring system. The use of well-defined interfaces, design patterns, and modular components ensures that the system meets the stringent requirements of availability, privacy, and data integrity.